

A 22 nm, 1540 TOP/s/W, 12.1 TOP/s/mm² in-Memory Analog Matrix-Vector-Multiplier for DNN Acceleration

I. A. Papistas, Stefan Cosemans, Bram Rooseleer, Jonas Doevenspeck, M.-H. Na, Arindam Mallik, Peter Debacker, Diederik Verkest
imec, Leuven, Belgium

Deep learning constitutes the state-of-the-art in machine learning, from data mining to computer vision and natural language processing. Energy-efficient matrix-vector multiplications (MVMs) [1] are key to bringing these capabilities to the edge, for both convolutional and recurrent neural networks. Since low precision multiplications with a limited level of noise can achieve relatively high accuracies [2], a compelling opportunity arises for mixed-signal hardware implementations to exploit in-memory computing paradigms [3–6]. In this work, a matrix-vector multiplier is presented operating on ternary weights, 7-bit input (6-bit magnitude and 1-bit sign), and 6-bit output activations. Notable improvements in energy efficiency and peak compute throughput per unit area are exhibited by measurements on a test chip based on the GLOBALFOUNDRIES® 22FDX® FD-SOI platform, thanks to a novel SRAM-based compute cell with low “on” current, pulse-width encoded input activations, a clamp-free summation line, and efficient amortization of the ADC and DAC energies.

The compute cell concept is illustrated in Fig. 1. It comprises two SRAM cells that store the ternary weight w ($w_-, 0, w_+$), two long-channel transistors in saturation that act as current limiters, and switches controlled by the cell weight and the pulse-width encoded input activations a_+ and a_- . Depending upon the sign of $a \times w$, the compute cell discharges either the positive (sum_+) or negative (sum_-) summation line proportionally to $|a \times w|$. The sign of the multiplication is determined by the ternary weight stored within the SRAM cells and by whether a pulse is applied on either activation lines. Rather than using two devices to regulate the multiplication magnitude and sign, a single transistor is utilized as a switch, where the SRAM cell is connected to the device gate and the complement of the activation pulse is applied on the device source. This results in an area and energy reduction of the compute cell, while the issue of a tail charge that occurs with a more naive, two-device implementation, is avoided.

The compute cells configuration per summation line is depicted in Fig. 2. Both sum_+ and sum_- are precharged to V_{DD} and then are discharged to some extent by the compute cells, according to the cell current I_{cell} and the activation pulse duration t_{act} . To regulate I_{cell} and t_{act} , an externally controllable gate bias $csbias$ and a programmable unit time t_{unit} are included for the current limiter and the pulse width encoded activations, equivalently. As the current limiter is in saturation, I_{cell} depends weakly upon the drain voltage connected to the summation line. Therefore, there is no requirement to clamp the summation line to a fixed voltage. Rather, the clamp is substituted by a simple precharge/discharge scheme instead.

The block-level diagram of the analog macro is shown in Fig. 3. 1024 activations are input to the macro and 512 6-bit words are generated per clock cycle. A pulse is generated on a_+ or a_- with the pulse width proportional to $|a|$. If $a = 0$, there is no activity on the DAC outputs, hence sparse activation vectors reduce energy consumption significantly. Subsequently, the voltage difference ΔV_{sum} on each summation line pair is sampled by the SAR-ADCs and converted to a 6-bit word. The measured transfer curves of the ADCs and DACs are also depicted in Fig. 3. On the left, good linearity and offset over the entire operating range is observed for the SAR-ADCs at both 0.8 V and 0.6 V. On the right, the linearity of the DACs is demonstrated for an increasing number of active weights, for activation levels between -63 and $+63$.

The result of a single MVM operation is illustrated in Fig. 4 for both 0.8 V and 0.6 V, using a synthetic workload of random weight matrices (with 30% 1, 30% -1 and 40% 0) and activation vectors (with 50% 0, 50% uniformly distributed between $-63, \dots, +63$). The quantization accuracy is $\sigma/step = 43\%$ at 0.8 V and 52% at 0.6 V, mainly attributed to the ADC operation. Such variation is tolerated by the network with proper training [2], while significant gains in energy can be achieved by relaxing the ADC specifications. A breakdown of the energy consumed for an MVM operation is depicted in Fig. 5. Even with 7- or 6-bit input and output precision, DAC and ADC energy is not negligible ($\sim 41\%$). Satisfactory energy efficiency is obtained if the ADC and DAC energy is amortized over several MVM operations.

In detail, the low “on” current (e.g. $\sim 0.85 \mu A$) attributed to the compute cell, the insensitivity of this current to $V_{sum, \pm}$, and the low mismatch ($2.3\% \sigma/\mu$ for $V_{csbias} = 0.6 V$) between the cells allow the connection of several cells in parallel; significantly amortizing the ADC energy. The use of pulse-width encoded activations enables the use of standard digital repeaters. Thus, DAC energy amortization is enabled.

The peak energy efficiency and throughput at 0.8 V and 0.6 V are also shown in Fig. 5 for different operating conditions. The listed figures assume the full 1024×512 array is used. Assuming 99% of the activations are non-zero, the energy efficiency at 0.8 V is 1050 TOP/s/W. At 0.6 V this is 1870 TOP/s/W, respectively. These energy efficiency figures correspond to a full ADC range output (maximum quantization accuracy). A reduced output range can be utilized, for example, in half range operation. The maximum operating frequency is 22.5 MHz at 0.8 V and 5.6 MHz at 0.6 V corresponding to a peak throughput of 23.6 TOP/s and 5.9 TOP/s, respectively.

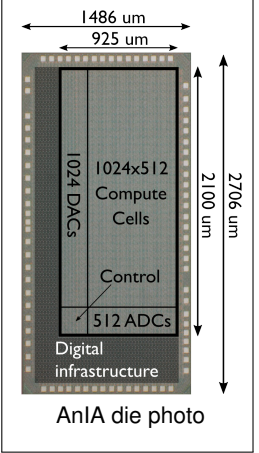
To benchmark the design, a ResNet-20 network is trained utilizing the CIFAR-10 dataset. The network is trained for ternary weights and 5-bit input and output activations. The baseline accuracy of the quantized network is 89.5% (FP baseline: 91.25%) including noise for the activations, weights, and accumulation. Two additional effects are included into the training process, the single-ended clipping (compute cell leaving the intended operating range) and the ADC differential clipping ($|\Delta V_{sum}| > \text{ADC operating range}$). Convolutional layers (18) are sequentially evaluated on-chip, however, fully-connected layers (2), batch normalization, and activation functions are evaluated on the host CPU. An accuracy of 88% is achieved with common settings (t_{unit} and $csbias$) for all layers.

A comparison to the state-of-the-art is given in Fig. 6 illustrating the performance of the design. The analog macro occupies an area of 1.94 mm^2 , including the compute cells, DACs, ADCs, and control. The test infrastructure of the matrix-vector multiplier supports up to 64 consecutive operations at speed, optionally in a loop. The energy consumed by the test infrastructure is not included in the reported energy numbers.

Summarizing, an analog in-memory matrix-vector-multiplier with ternary weights, 7-bit input activations, and 6-bit accumulation is presented in this work. State-of-the-art efficiency and throughput per unit area are shown, supported by silicon measurements of the prototype test chip designed in a 22 nm process node. Four factors contributed to these results: (1) a novel SRAM-based compute cell with low current summation element, (2) pulse-width encoded input activations, (3) a clamp-free summation line, and (4) a large array, enabled by the low current levels and digital repeaters for the pulse-width activations. Efficient amortization of the ADC and DAC energies is achieved, while an accuracy of 88% is exhibited on the CIFAR-10 dataset for a quantized ResNet-20 network.

References

- [1] J. Choi *et al.*, “Accurate and Efficient 2-bit Quantized Neural Networks,” *SysML*, March 2019.
- [2] B.-E. Verhoef *et al.*, “FQ-Conv: Fully Quantized Convolution for Efficient and Accurate Inference,” *arXiv preprint*, 2019.
- [3] S. Yin *et al.*, “XNOR-SRAM: In-Memory Computing SRAM Macro for Binary/Ternary Deep Neural Networks,” *JSSC*, June 2020.
- [4] H. Valavi *et al.*, “A 64-Tile 2.4-Mb In-Memory-Computing CNN Accelerator Employing Charge-Domain Compute,” *JSSC*, June 2019.
- [5] D. Bankman *et al.*, “An Always-On 3.8 μJ /86% CIFAR-10 Mixed-Signal Binary CNN Processor With All Memory on Chip in 28-nm CMOS,” *JSSC*, Jan. 2019.
- [6] Q. Dong *et al.*, “15.3 A 351TOPS/W and 372.4 GOPS Compute-in-Memory SRAM Macro in 7nm FinFET CMOS for Machine-Learning Applications,” *ISSCC*, Feb 2020.



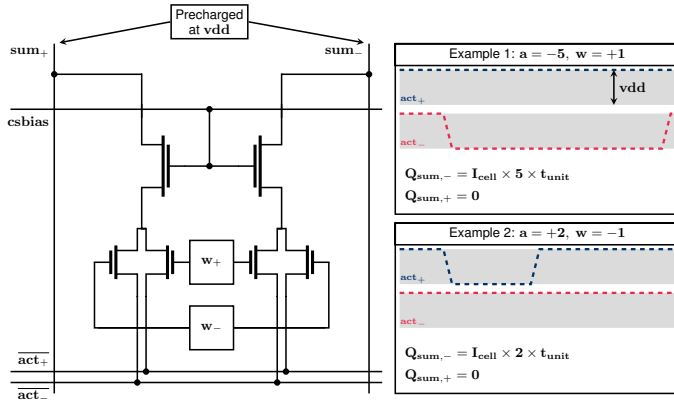


Fig. 1. Compute cell concept. The weights w are stored in two SRAM cells, two long-channel transistors are utilized as current limiters, and switches determine whether to discharge sum_+ or sum_- based on the sign of input activation act and weight w .

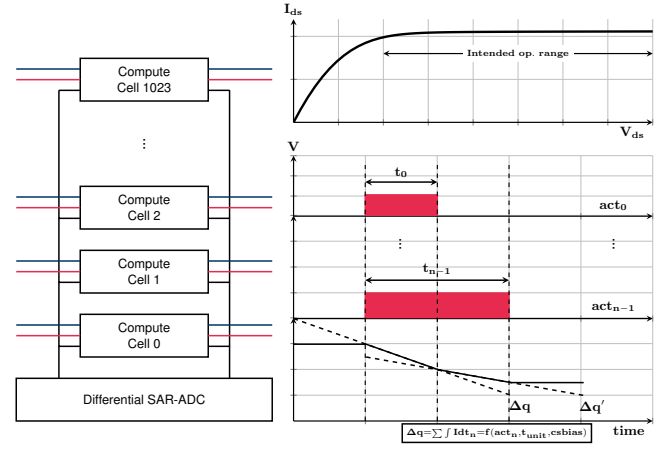


Fig. 2. 1024 cell contributions are accumulated on the summation line pair. A simple precharge-discharge scheme is enabled thanks to the high output impedance of the cell current source and the low on current. I_{ds} and V_{ds} refer to the compute cell long channel transistor operating as a current limiter. The operating range of the compute cell, is described by $V_{sum,\pm} \geq V_{cs_{bias}} - V_t$.

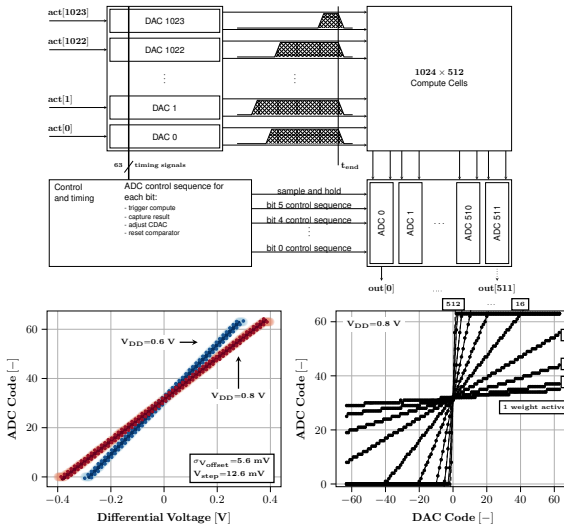


Fig. 3. The analog MVM macro is illustrated at the top. The linearity of the 6-bit ADCs is depicted on the left for 0.8 V and 0.6 V supplies, including the 1σ variation, the $\sigma_{V_{offset}}$ and the V_{step} . The DACs operation at 0.8 V supply is on the right for an increasing number of active weights per summation line.

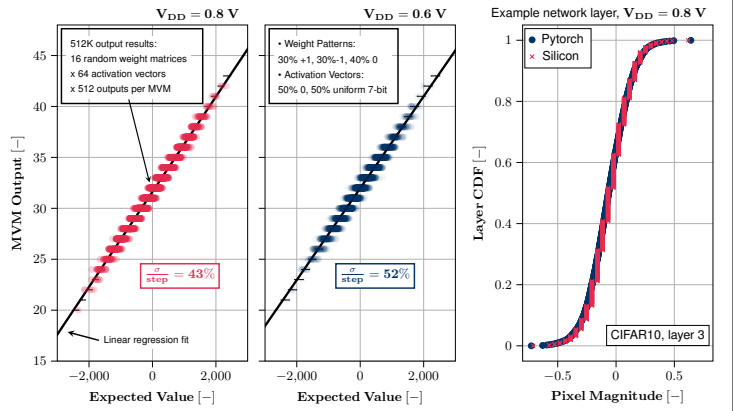


Fig. 4. The outcome of the MVM operation is illustrated at both 0.8 V and 0.6 V supply voltages in the first two graphs, respectively. Ideal 6-bit quantization would result in $\sigma/step = 1/\sqrt{12} \approx 29\%$. The complete MVM operation (DAC, compute cell, accumulation, ADC) is included in these graphs and it behaves linearly. A synthetic dataset is utilized to show the linearity of the analog macro. A CIFAR10 layer is depicted in the third graph, where the computation on silicon is shown in red, compared to the software computation in pytorch shown in blue.

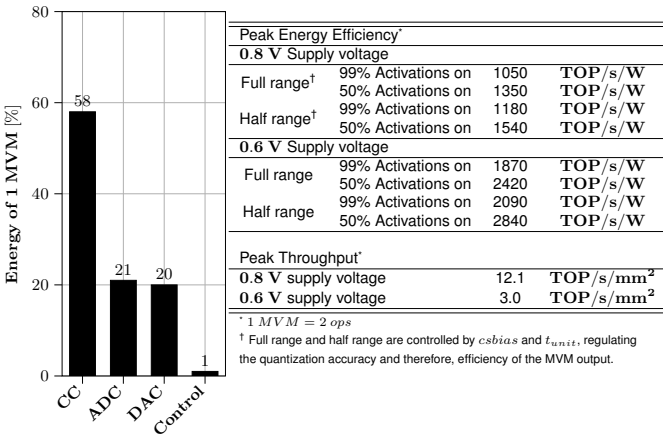


Fig. 5. The energy efficiency for different operating conditions is listed in the table, while an energy breakdown per component within the analog macro is also included. A significant portion of the energy is utilized on the compute cells due to the long summation and activation lines that span the entire array. The energy utilized for each multiply and accumulate (MAC) within the MVM is $\sim 1 fJ/op$ at 0.8 V for the typical operating condition. For reference, 1 MAC is counted as 2 ops

Table A

	Yin [3] JSSC Jan20	Valavi [4] JSSC Jun19	Bankman [5] JSSC Jan19	Dong [6] ISSCC20	This work MVM Only
Process Node	65 nm	65 nm	28 nm	7 nm	GF 22FDX
Area	0.11 mm²	12.6 mm²	1.6 mm²	0.0032 mm²	1.94 mm²
Memory Size	16 Kb	2.4 Mb	2.6 Mb	4 kb	1 Mb
Precision					
Weights	Ternary	1	1	4	Ternary
Input Activations	1	1	1	4	6b + sign
Output Activations	3.46	1	1	4	6
Peak Performance @ V_{core}					
Throughput	0.61 TOP/s	18.9 TOP/s	0.48 TOP/s	0.37 TOP/s	0.8 V: 23.5 TOP/s 0.6 V: 5.8 TOP/s
Area Efficiency	5.5 TOP/s/mm²	1.5 TOP/s/mm²	0.3 TOP/s/mm²	116.2 TOP/s/mm²	12.1 TOP/s/mm²
Energy Efficiency	403 TOP/s/W	866 TOP/s/W	772 TOP/s/W	351 TOP/s/W	1050 TOP/s/W

Table B

Component	Area	Units
Compute Cells†	1.35	mm²
ADCs	0.11	mm²
DACs	0.22	mm²
Control	0.03	mm²
Decaps	0.23	mm²

† Including the area of repeaters for the activation line.

Fig. 6. A comparison to the state of the art is shown in Table A, listing significant figures of merit for similar designs. An edge-class energy efficiency and energy per unit area are achieved, especially considering the implemented input and output bit-widths, compared to the state-of-the-art. A summary of the design is listed in Table B.